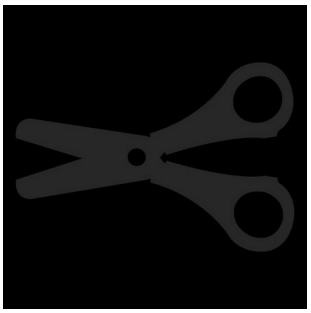
Universidad Europea de Madrid

Grado Superior DAW · Curso 2025/2026



PROYECTO INTEGRADOR 1DAW

Los Hijos Del Compilador

Diego Muñoz Avilés
David Martínez del Río
Javier García Pablos
Roberto Canto Pedrera

Resumen

El presente documento constituye la memoria final del Proyecto Integrador desarrollado durante el curso 2025/2026 por el equipo Los Hijos Del Compilador, formado por estudiantes de primer curso del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW) de la Universidad Europea de Madrid. El proyecto consiste en el diseño, implementación y documentación de una aplicación de escritorio para la gestión integral del taller de Edna Moda, un estudio ficticio especializado en la confección de trajes para superhéroes y supervillanos, propuesto como caso práctico interdisciplinar que integra los contenidos de los módulos de Programación, Bases de Datos y Entornos de Desarrollo.

La aplicación se ha construido en **Java 17** siguiendo estrictamente el patrón arquitectónico **Modelo-Vista-Controlador** (MVC), con interfaz gráfica de usuario desarrollada en Java Swing y persistencia de datos mediante **MySQL 8** a través de la API **JDBC**. Implementa un sistema CRUD completo sobre las entidades Empleados, Clientes, Trajes, Talleres y Citas, un sistema de autenticación con hash **SHA-256** y control de acceso por roles (Aprendiz, Oficial y Maestro), así como reglas de negocio avanzadas (solapes horarios, límite de aprendices por cita y regla bonus de separación temporal entre clientes héroes y villanos).

El desarrollo se ha planificado en cuatro sprints de dos semanas cada uno, aplicando la metodología ágil **SCRUM**, con apoyo en GitHub para el control de versiones, Trello para la gestión de tareas y **JUnit 5** para la validación del código mediante pruebas unitarias. La documentación técnica se ha generado con JavaDoc y el manual de usuario se ha publicado en la wiki del repositorio de GitHub. El resultado final es un producto software estable, documentado y completamente funcional, que cubre la totalidad de los requisitos definidos en el enunciado del proyecto.

Palabras clave: Java, Swing, MySQL, **JDBC**, MVC, **DAO**, JUnit, JavaDoc, **SCRUM**, GitHub, gestión de citas, autenticación, patrón de diseño, control de acceso por roles.

1. Introducción

El Proyecto Integrador del primer curso del ciclo de Desarrollo de Aplicaciones Web tiene como propósito principal aplicar de forma conjunta los conocimientos adquiridos en los tres módulos profesionales cursados durante el año: Programación, Bases de Datos y Entornos de Desarrollo. Con este enfoque transversal se persigue que el alumnado experimente, en un contexto lo más realista posible, las fases completas del desarrollo de un producto software profesional: desde el análisis de requisitos y el diseño arquitectónico, hasta la implementación, pruebas y entrega documentada del sistema.

El enunciado del proyecto propone como caso de estudio el taller de Edna Moda, un estudio ficticio dedicado al diseño y confección de trajes para superhéroes y supervillanos. Este escenario, inspirado en el universo cinematográfico de Pixar, exige construir una aplicación de escritorio que permita gestionar la operativa diaria del taller: registrar a los clientes (héroes y villanos), administrar sus trajes en los distintos estados de fabricación, organizar los talleres físicos del estudio (salas de diseño, costura y pruebas en distintas ciudades) y, sobre todo, planificar las citas en las que un empleado responsable atiende a un cliente acompañado, en su caso, de hasta dos aprendices.

El equipo Los Hijos Del Compilador, formado por Diego Muñoz Avilés, David Martínez del Río, Javier García Pablos y Roberto Canto Pedrera, ha asumido el desarrollo del proyecto bajo una organización ágil basada en **SCRUM**, dividiendo el trabajo en cuatro sprints de dos semanas. En cada sprint se han abordado fases específicas del desarrollo: análisis y diseño funcional (Sprint 1), modelado relacional, creación de la base de datos y prototipo de la interfaz (Sprint 2), implementación de las capas de Modelo y Control (Sprint 3) y, finalmente, refinamiento, pruebas, integración y documentación del producto completo (Sprint 4).

Este documento recoge de forma exhaustiva el conjunto del trabajo realizado a lo largo del proyecto: las decisiones de diseño adoptadas, las tecnologías empleadas, la arquitectura del sistema, los problemas encontrados y la forma en la que se han resuelto. Asimismo, incluye los anexos exigidos por la rúbrica del proyecto: el listado completo de requisitos funcionales y no funcionales, la guía de uso de la aplicación y los diagramas técnicos (casos de uso, clases UML y modelo entidad-relación) que sustentan el diseño.

2. Objetivos

2.1. Objetivo general

Diseñar, desarrollar, documentar y entregar una aplicación de escritorio funcional, estable y mantenible que permita la gestión integral del taller de Edna Moda, cubriendo la totalidad de los requisitos descritos en el enunciado del Proyecto Integrador y aplicando de manera práctica los contenidos curriculares de los módulos de Programación, Bases de Datos y Entornos de Desarrollo.

2.2. Objetivos específicos

El objetivo general se ha desglosado en una serie de objetivos específicos que guían el trabajo realizado en cada sprint:

- **Analizar funcionalmente el problema planteado mediante un diagrama de casos de uso.**
- **Diseñar un modelo de clases UML.**

- **Construir un modelo entidad-relación.**
- **Implementar la aplicación en Java 17 aplicando el patrón Modelo-Vista-Controlador (MVC).**
- **Desarrollar la interfaz gráfica con Java Swing.**
- **Integrar la persistencia mediante JDBC aplicando el patrón DAO (Data Access Object).**
- **Implementar un sistema de autenticación seguro con SHA-256.**
- **Garantizar el cumplimiento de las reglas de negocio.**
- **Verificar la calidad del código mediante pruebas unitarias con JUnit 5.**
- **Aplicar la metodología SCRUM.**
- **Elaborar la memoria técnica del proyecto.**

3. Tecnologías utilizadas

La selección de tecnologías se ha guiado por dos criterios: (1) garantizar la cobertura curricular de los tres módulos del ciclo formativo y (2) asegurar la disponibilidad de herramientas robustas, gratuitas y ampliamente utilizadas en el sector profesional.

3.1. Lenguaje y entorno de ejecución

El núcleo del proyecto se desarrolla en **Java 17** LTS, versión soportada a largo plazo que aporta mejoras significativas en rendimiento, expresividad del lenguaje (registros, switch expressions, pattern matching) y compatibilidad con el ecosistema actual de bibliotecas. La interfaz gráfica se construye con Java Swing, biblioteca incluida en el JDK que ofrece un control fino sobre la composición de ventanas, el modelo de eventos y los componentes de tabla (JTable) y de cuadros de diálogo (JOptionPane) utilizados para la introducción y visualización de datos.

3.2. Base de datos

La persistencia se ha implementado sobre **MySQL 8**.0, sistema relacional de código abierto que cumple los requisitos del módulo de Bases de Datos. La conexión desde la aplicación Java se realiza mediante el conector oficial mysql-connector-j 9.6.0 sobre la API **JDBC**, ubicado en el directorio lib/ del proyecto y referenciado desde el classpath de Eclipse. La base de datos utiliza la codificación utf8mb4 con la collation utf8mb4_spanish_ci para soportar adecuadamente los caracteres del castellano.

3.3. Herramientas de desarrollo y colaboración

La siguiente tabla resume las tecnologías y herramientas empleadas, indicando la versión utilizada y la finalidad en el proyecto:

Tecnología	Versión	Finalidad
Java JDK	17 LTS	Lenguaje de programación principal.
	Incluido en JDK	Construcción de la interfaz gráfica (Vista).
MySQL	8.0	Sistema gestor de base de datos relacional.
mysql-connector-j	9.6.0	Driver **JDBC** para conexión con MySQL.

Tecnología	Versión	Finalidad
Eclipse IDE	2024-12	Entorno de desarrollo integrado.
Git	2.45+	Sistema de control de versiones distribuido.
	—	Hosting del repositorio remoto y wiki del manual.
Trello	—	Tablero Kanban para gestión de Sprints.
JUnit	5.10	Framework de pruebas unitarias.
	Incluido en JDK	Generación de documentación técnica.
PlantUML	1.2024	Generación de diagramas UML.
draw.io / dbdiagram	—	Diagramas E-R y de clases.
Markdown	—	Manual de usuario en la wiki de GitHub.

4. Desarrollo e implementación

4.1. Especificaciones funcionales (qué hace la aplicación)

Antes de entrar en el detalle técnico de la arquitectura, se describe a alto nivel qué hace exactamente la aplicación entregada. Esta visión funcional resume las capacidades del sistema y sirve de marco a los diagramas de casos de uso que se presentan a continuación.

La aplicación implementa una herramienta de escritorio para gestionar la operativa diaria del taller de Edna Moda. El usuario se autentica con apodo y contraseña, y a partir de ese momento la interfaz se adapta a su rol (Aprendiz, Oficial o Maestro), mostrando solo las acciones que tiene permitidas. Las grandes áreas funcionales son las siguientes:

Gestión de citas

- Aprendiz: consulta el listado de citas en las que figura como aprendiz asignado y puede abrir el detalle de cada una para ver cliente, traje, taller, día, hora y compañeros.
- Oficial: visualiza todas las citas del sistema, crea nuevas citas (en las que él mismo queda como responsable) y modifica las suyas. Puede asignar hasta 2 aprendices por cita.
- Maestro: además de lo anterior, modifica cualquier cita (incluyendo las creadas por otros), borra citas y puede asignar como responsable a otro oficial distinto al actual.

Toda creación o modificación de cita aplica de forma automática tres reglas de negocio: máximo 2 aprendices por cita, prohibición de solapes horarios en el mismo taller y, como regla bonus, separación mínima de una hora entre cita de héroe y de villano en el mismo taller y día.

Gestión de clientes y trajes

- Aprendiz y Oficial: consultan el listado de clientes (superhéroes y supervillanos), abren su ficha y ven los trajes asociados.
- Oficial: puede cambiar el estado de un traje (Diseño → Costura → Pruebas) según avanza el trabajo.

- Maestro: realiza el CRUD completo sobre clientes y trajes (alta, modificación y baja). El alta de cliente permite definir un traje principal en el mismo paso.

Gestión de talleres

- Oficial y Maestro: consultan los talleres disponibles (Milán, París, Madrid, Nueva York, Berlín…) con su tipo (Diseño, Costura o Pruebas).
- Maestro: realiza el CRUD completo sobre talleres (alta, modificación de nombre/tipo y baja).

Gestión de empleados

- Operación reservada al Maestro: alta, modificación y baja de empleados, asignación de categoría (Aprendiz/Oficial/Maestro) y restablecimiento de la contraseña al valor por defecto.
- La aplicación protege al Maestro autenticado para que no pueda borrarse a sí mismo ni rebajar su categoría, y garantiza que siempre quede al menos un Maestro activo en la plantilla.
- El apodo "EdnaM" (propietaria del taller) está blindado: no puede borrarse, ni cambiar de apodo, ni dejar de ser Maestro.

Diagramas de casos de uso

Las siguientes figuras formalizan estas capacidades como diagramas UML de casos de uso. Para facilitar la lectura, en lugar de un único diagrama global, se ha optado por presentar un diagrama por actor. El Maestro hereda todas las capacidades del Oficial, y este a su vez las del Aprendiz. Los casos CRUD se desglosan en sus operaciones individuales (Consultar, Crear, Modificar, Borrar) para reflejar con precisión la funcionalidad real de la aplicación.

El primer diagrama recoge los casos de uso del Aprendiz, el rol con menor nivel de privilegios. Sus capacidades son de consulta: ver únicamente las citas en las que él figura como aprendiz asignado, abrir el detalle de cada una de ellas (relación «include») y consultar el listado de clientes y trajes. Adicionalmente, como cualquier empleado del sistema, puede cambiar su propia contraseña.

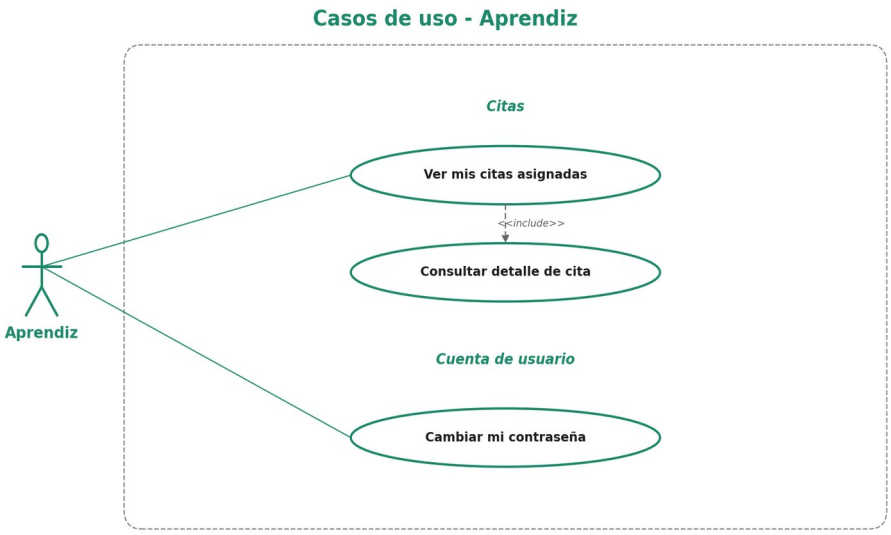


Figura 1. Casos de uso del actor Aprendiz.

El segundo diagrama muestra el rol del Oficial, que extiende las capacidades del Aprendiz. El Oficial puede ver el calendario completo de citas, crear nuevas (en las que él mismo queda como responsable) y modificar las suyas, asignar hasta dos aprendices, consultar los talleres disponibles y cambiar el estado de un traje conforme avanza el trabajo (Diseño → Costura → Pruebas).

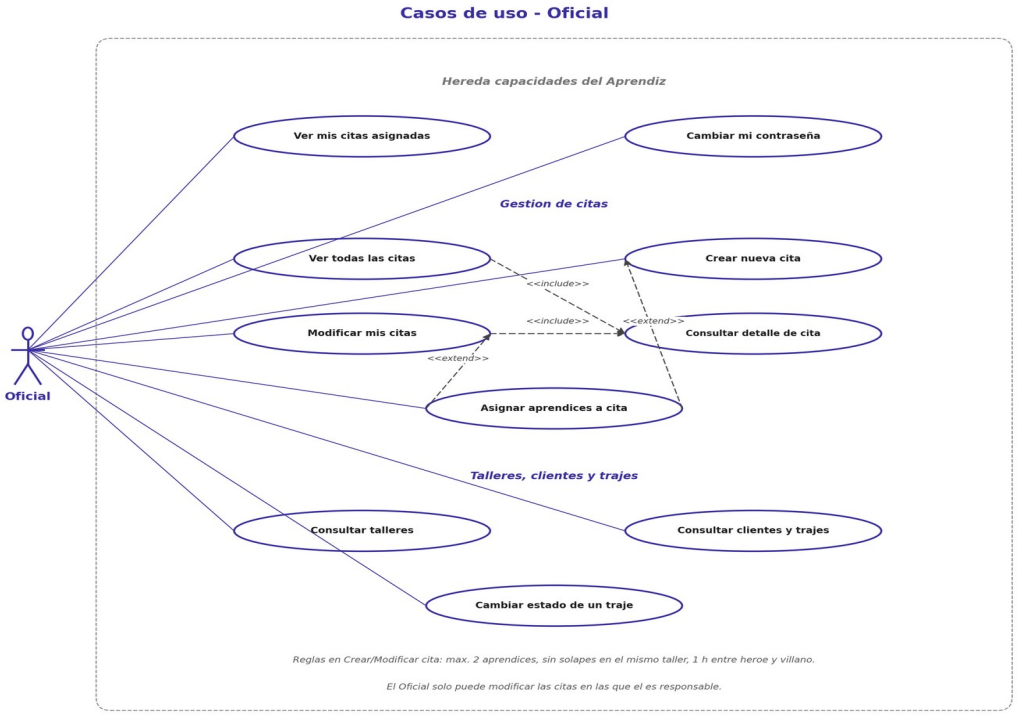


Figura 2. Casos de uso del actor Oficial.

El tercer diagrama corresponde al Maestro, el rol con acceso total al sistema. Además de heredar las capacidades del Oficial, el Maestro puede modificar cualquier cita (no solo las propias), asignar como responsable a otro Oficial, borrar citas y, sobre todo, ejecutar las operaciones CRUD completas sobre Clientes, Trajes, Talleres y Empleados. También dispone de la capacidad exclusiva de restablecer la contraseña de cualquier empleado al valor por defecto.

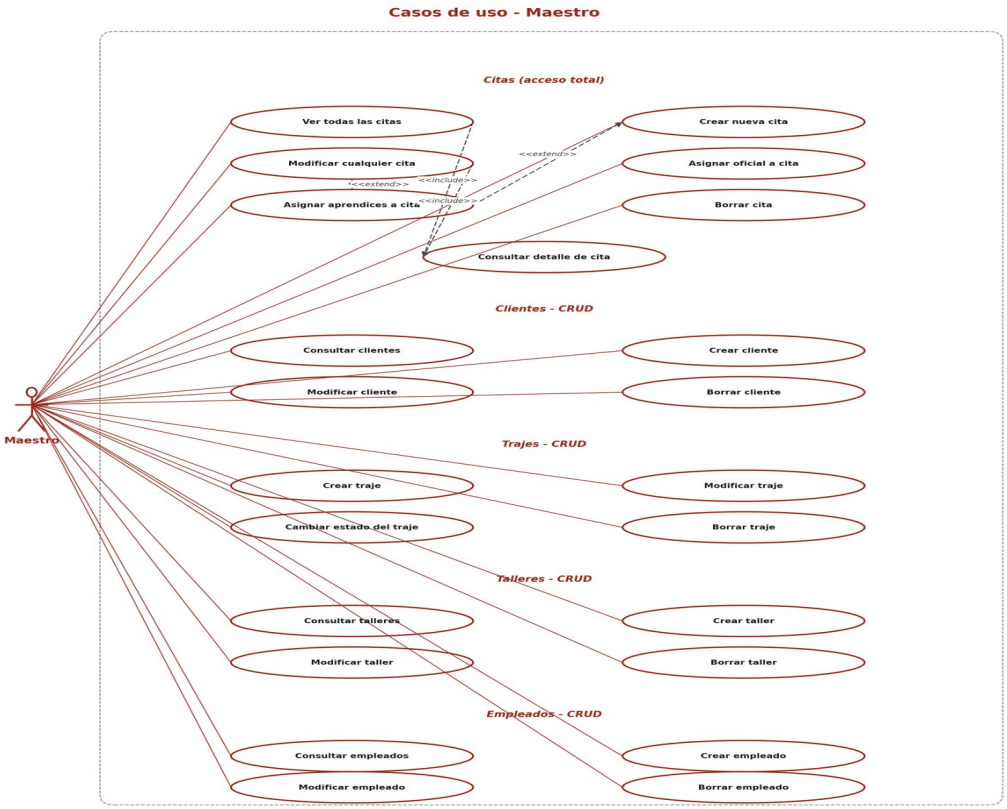


Figura 3. Casos de uso del actor Maestro.

4.2. Arquitectura general (patrón MVC)

La aplicación sigue de manera estricta el patrón arquitectónico **Modelo-Vista-Controlador**, separando las responsabilidades en tres capas claramente diferenciadas. Esta separación favorece la mantenibilidad, la extensibilidad y la testabilidad del código, permitiendo modificar una capa sin afectar al resto.

4.3. Capa Modelo

La capa Modelo está compuesta por cinco clases POJO (Plain Old Java Object) que representan las entidades fundamentales del dominio. Cada clase encapsula los atributos del enunciado y expone los getters y setters necesarios, junto a constructores convenientes para el uso desde los **DAO** y desde la lógica de negocio. Los valores restringidos (categoría del empleado, estado del traje, tipo de sala y tipo de cliente) se modelan como enum anidados con métodos fromTexto() que permiten la conversión bidireccional con los valores almacenados en MySQL.

La clase Cita, núcleo del dominio, mantiene la lista de aprendices como List<Integer> (en coherencia con la tabla N:M cita_aprendices) y conoce su propia restricción de negocio mediante la constante MAX_APRENDICES = 2, validada en su método anadirAprendiz(). Las clases Modelo también incluyen operaciones útiles para el control como Empleado.puedeCrearCitas(), Empleado.esMaestro() y Cita.getHoraFin(), lo que mejora la legibilidad de la capa Controlador.

4.4. Capa Vista

La capa Vista agrupa las ventanas Swing de la aplicación. Sigue el principio de pasividad: las vistas no contienen lógica de negocio, solo presentación y captura de eventos, que se delegan al controlador correspondiente. Las ventanas construidas son:

- LoginVista: pantalla inicial con campos para apodo y contraseña.
- VentanaPrincipal: ventana principal con barra de menú dinámica que habilita o deshabilita opciones en función del rol del empleado autenticado.
- CitasVista: gestión completa del calendario de citas, con tabla maestra de citas existentes y formulario lateral para alta, modificación y consulta.
- ClientesVista: ABM (alta-baja-modificación) de clientes.
- TalleresVista: ABM de talleres del estudio, accesible solo al Maestro.
- EmpleadosVista: ABM de empleados, accesible solo al Maestro.

Todas las vistas exponen getters de sus componentes interactivos (botones, tablas, campos de texto), lo que permite a los controladores registrar listeners sin acoplarse a la implementación interna del Swing.

4.5. Diseño de la base de datos

Este apartado reúne todo el diseño de datos del proyecto: el diagrama entidad-relación elaborado en el Sprint 2, el proceso de normalización aplicado, el diagrama de clases UML que sirvió de puente entre el modelo conceptual y el código Java, y por último el esquema físico resultante implementado en MySQL.

4.5.1. Diagrama entidad-relación

El modelo E-R identifica las entidades del dominio (Empleado, Cliente, Traje, Taller, Cita) y sus relaciones: un cliente tiene 1..* trajes, una cita reserva un taller para atender a un cliente sobre un traje concreto con un empleado responsable, y una cita puede tener 0..2 aprendices asignados (relación N:M materializada en la tabla cita_aprendices).

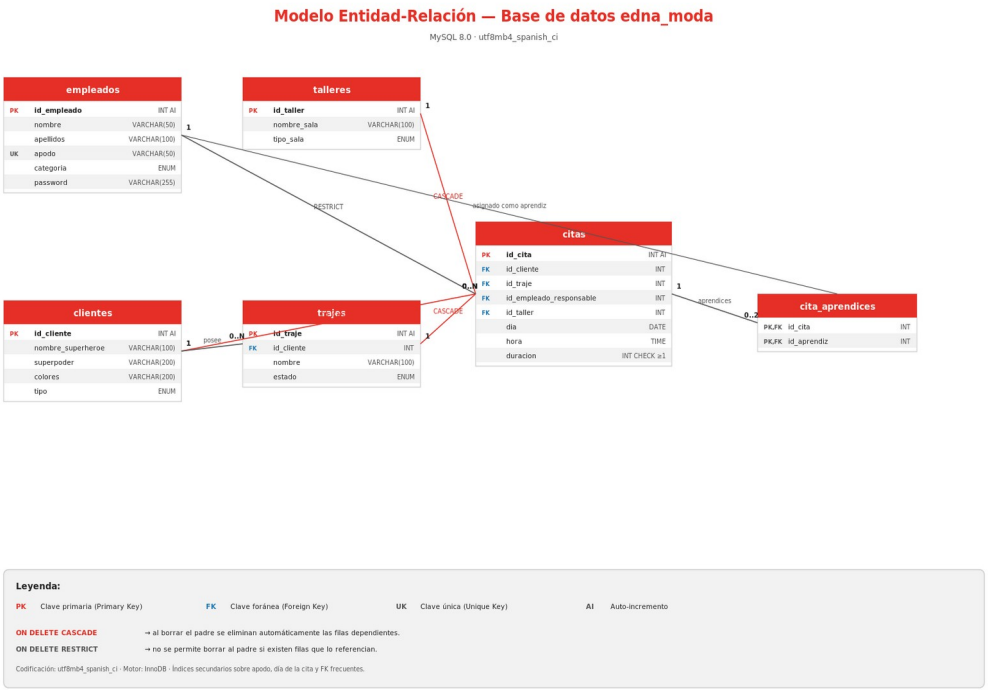


Figura 4. Diagrama entidad-relación.

4.5.2. Normalización

El esquema se ha normalizado hasta la **3ª Forma Normal** (3FN), siguiendo el procedimiento clásico:

- **1FN:** todos los atributos contienen valores atómicos. No existen campos repetidos ni listas dentro de una columna. Por ejemplo, los aprendices de una cita no se guardan como cadena "1,3" en la tabla citas, sino que se desnormalizan en filas independientes de cita_aprendices.
- **2FN:** todos los atributos no clave dependen funcionalmente de la totalidad de la clave primaria. Esto es relevante en cita_aprendices, cuya PK es compuesta (id_cita, id_aprendiz): la tabla no contiene atributos adicionales que dependan solo de uno de los dos campos.
- **3FN:** no existen dependencias transitivas entre atributos no clave. Por ello, los datos del cliente no se replican en cada cita (solo se guarda id_cliente) y los datos del taller no se replican en cada cita (solo id_taller). Tampoco se almacena la categoría del empleado en la tabla citas, sino que se consulta a través de id_empleado_responsable.

La aplicación rigurosa de la 3FN elimina la redundancia, simplifica las operaciones de actualización (un cambio de nombre de un cliente o de un taller se realiza en una sola fila) y previene anomalías de inserción, modificación y borrado.

4.5.3. Diagrama de clases UML

El diagrama de clases UML, refinado durante el Sprint 3, traduce el modelo conceptual al lenguaje de la programación orientada a objetos. Describe las clases del modelo (Empleado, Cliente, Traje, Taller, Cita), sus atributos, operaciones, visibilidad y multiplicidades, así como los enumerados del dominio (Categoria, Estado, TipoSala, Tipo).

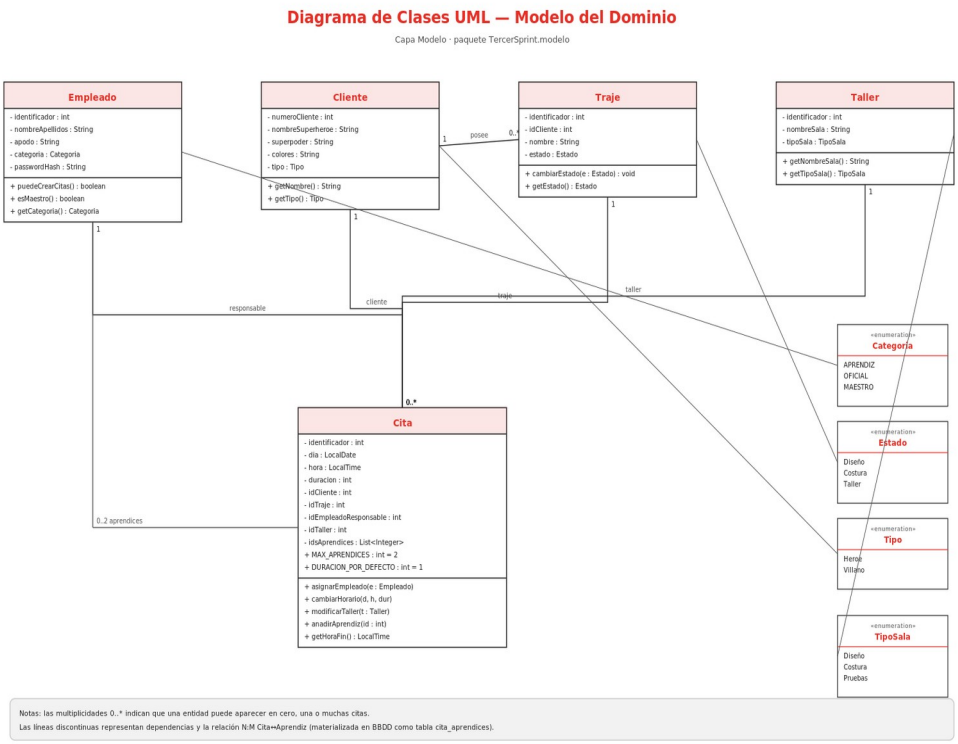


Figura 5. Diagrama de clases UML.

4.5.4. Esquema físico

La base de datos edna_moda implementa el modelo E-R sobre **MySQL 8**. Está formada por seis tablas: empleados, clientes, trajes, talleres, citas y la tabla intermedia cita_aprendices que materializa la relación N:M entre citas y aprendices.

Tabla	PK	Descripción
empleados	id_empleado	Plantilla del taller (Aprendiz / Oficial / Maestro).
clientes	id_cliente	Superhéroes y supervillanos (tipo: Héroe/Villano).
trajes	id_traje	Trajes asociados a un cliente con estado de fabricación.
talleres	id_taller	Salas del estudio (Diseño / Costura / Pruebas).
citas	id_cita	Reserva de un taller para atender a un cliente.
cita_aprendices	(id_cita, id_aprendiz)	Aprendices asignados a una cita (máx. 2).

Todas las claves foráneas declaran integridad referencial mediante FOREIGN KEY, y aquellas que admiten borrado en cascada utilizan ON DELETE CASCADE (es el caso de trajes asociados a un cliente o de aprendices vinculados a una cita). Las claves que protegen datos críticos utilizan ON DELETE RESTRICT (por ejemplo, no se puede borrar un empleado que sea responsable de alguna cita).

Esquema resumido:

```
empleados        (id_empleado PK, nombre, apellidos, apodo UK, categoria, password)
clientes         (id_cliente PK, nombre_superheroe, superpoder, colores, tipo)
trajes           (id_traje PK, id_cliente FK, nombre, estado)
talleres         (id_taller PK, nombre_sala, tipo_sala)
```

```
citas            (id_cita PK, id_cliente FK, id_traje FK, id_empleado_responsable
FK,
                  id_taller FK, dia, hora, duracion)
cita_aprendices  (id_cita FK, id_aprendiz FK)  -- PK compuesta
```

4.6. Capa Controlador y acceso a datos (DAO)

La capa de Control se subdivide intencionadamente en dos niveles:

- Nivel **DAO** (Data Access Object): clases responsables del acceso a la base de datos. Son las únicas que contienen sentencias SQL y se comunican siempre mediante **PreparedStatement** para prevenir inyección SQL. Existe un DAO por cada entidad principal: EmpleadoDAO, ClienteDAO, TrajeDAO, TallerDAO y CitaDAO.
- Nivel Controlador: clases que orquestan los **DAO** y aplican las reglas de negocio. Son las que invocan las vistas. Esta separación mantiene los DAO limpios y reutilizables y permite testar las reglas de negocio de forma independiente.

La clase ConexionBD centraliza la obtención de conexiones **JDBC** mediante bloques **try-with-resources**, garantizando la liberación automática de recursos. Por su parte, SesionActual implementa el patrón **Singleton** para mantener el empleado autenticado a lo largo de la ejecución.

Las operaciones que afectan a varias tablas — por ejemplo, la creación o modificación de una cita, que escribe en citas y cita_aprendices — se envuelven en transacciones **JDBC** mediante **setAutoCommit(false)**, commit() y rollback(), garantizando la atomicidad de las operaciones.

4.7. Reglas de negocio implementadas

Las reglas de negocio se concentran en los controladores, evitando contaminar la capa de acceso a datos. Las más relevantes son:

- **Autenticación segura:** la contraseña introducida se transforma con **SHA-256** y se compara con el hash almacenado.
- **Control de acceso por roles:** el Aprendiz solo visualiza las citas en las que participa; el Oficial puede crear citas y modificar las propias; el Maestro tiene acceso total.
- **Límite de aprendices por cita:** como máximo dos aprendices pueden asignarse a la misma cita (constante Cita.MAX_APRENDICES).
- **Solapes horarios:** el método Cita**DAO**.listarPorTallerYDia(idTaller, dia) permite al controlador detectar conflictos antes de persistir una nueva cita.
- **Regla bonus héroe/villano:** cuando un héroe y un villano coinciden en el mismo taller y día, debe existir un margen mínimo de 60 minutos entre sus citas.
- **Duración mínima de cita:** las citas tienen una duración mínima de 1 hora, validada tanto en la base de datos (CHECK duracion >= 1) como en el controlador.

4.8. Pruebas unitarias con JUnit

Durante el Sprint 4 se ha desarrollado una batería de pruebas unitarias con **JUnit 5** que cubre las clases más críticas del sistema. Las pruebas se organizan en el directorio test/ siguiendo la misma estructura de paquetes que el código de producción, y se han clasificado en tres bloques:

- Pruebas de modelo: validan las restricciones implementadas en las clases POJO (por ejemplo, el límite de aprendices de la clase Cita o las transiciones permitidas del enum Estado del traje).

- Pruebas de **DAO**: comprueban las operaciones CRUD utilizando una base de datos de pruebas. Se utilizan métodos @BeforeEach y @AfterEach para garantizar el aislamiento entre tests.
- Pruebas de controlador: verifican el cumplimiento de las reglas de negocio (rechazo de citas con más de dos aprendices, autenticación con credenciales incorrectas, prohibición de modificar citas ajenas para un Oficial, etc.).

Las pruebas han permitido detectar y corregir varias incidencias antes de la integración final, en particular un caso de bucle vacío en el manejo de aprendices y un problema de formato de fecha al persistir las citas.

4.9. Documentación JavaDoc

Todas las clases públicas del proyecto se han documentado con JavaDoc siguiendo las convenciones estándar: encabezado descriptivo, etiquetas @author, @version, @since, descripción de cada método, parámetros (@param), valor de retorno (@return) y excepciones lanzadas (@throws). La documentación se ha generado y publicado en la carpeta docs/ del repositorio, y queda accesible offline para cualquier desarrollador que clone el proyecto.

5. Metodología

5.1. SCRUM aplicado al proyecto

El equipo ha adoptado la metodología ágil **SCRUM** como marco de trabajo principal. Se han definido sprints de dos semanas, con los siguientes eventos clave en cada uno:

- **Sprint Planning:** reunión inicial en la que se priorizan las historias del backlog y se reparten las tareas entre los miembros del equipo.
- **Daily SCRUM:** breve sincronización (sin reunión formal, mediante mensajes diarios en el canal del equipo) en la que cada miembro indica qué hizo el día anterior, qué hará hoy y qué obstáculos encuentra.
- **Sprint Review:** presentación de los entregables completados al final del sprint, validando con el equipo (y con la profesora cuando procede) que los objetivos se han cumplido.
- **Sprint Retrospective:** análisis de qué ha ido bien, qué ha ido mal y qué se puede mejorar en el siguiente sprint.

5.2. Resumen de Sprints

Sprint	Fechas	Foco principal	Entregables clave
#1	24-feb a 09-mar 2026	Análisis y diseño funcional	Diagrama de casos de uso, esqueleto MVC inicial.
#2	10-mar a 06-abr 2026	Prototipo Vista + modelo relacional	Ventanas Swing (Login, Principal, Citas, Clientes) + BD.
#3	07-abr a 20-abr 2026	Implementación de las capas Modelo y Control	Clases POJO, ConexionBD, **DAO**s.
#4	21-abr a 18-may 2026	Refinamiento, pruebas, documentación y entrega	Conexión definitiva Vista-Controlador, pruebas JUnit,

Sprint	Fechas	Foco principal	Entregables clave
			JavaDoc.

5.3. Herramientas de gestión y control de versiones

El equipo ha trabajado de forma colaborativa apoyándose en las siguientes herramientas:

- **GitHub** como repositorio remoto. La rama main contiene siempre código estable. Cada historia de usuario se desarrolla en una rama feature/ que se integra mediante Pull Request tras revisión por al menos otro miembro del equipo.
- **Trello** como tablero Kanban del backlog del proyecto. Las tareas se distribuyen en columnas To Do, In Progress, Review y Done.
- **GitHub Wiki** para la publicación del manual de usuario en Markdown.
- Reuniones presenciales y online los lunes y miércoles para sincronizar el avance y resolver bloqueos.

6. Resultados y conclusiones

Al término del Sprint 4 el equipo dispone de una aplicación de escritorio completamente funcional que cubre la totalidad de los requisitos planteados en el enunciado del Proyecto Integrador. La aplicación implementa un sistema CRUD completo sobre las cinco entidades del dominio, un mecanismo robusto de autenticación con hash **SHA-256** y un control de acceso por roles que respeta estrictamente la jerarquía Aprendiz–Oficial–Maestro descrita en el enunciado. Las reglas de negocio más complejas (límite de aprendices, solapes horarios, regla bonus héroe/villano) se han implementado y verificado mediante pruebas unitarias.

Desde el punto de vista técnico, el proyecto ha permitido afianzar de forma práctica los principales contenidos curriculares del primer curso del ciclo: programación orientada a objetos con Java, diseño y consulta de bases de datos relacionales, uso profesional de un entorno integrado de desarrollo, control de versiones con Git, aplicación rigurosa del patrón MVC y separación de responsabilidades, y prácticas de documentación y pruebas que el equipo considera valiosas a futuro.

Desde el punto de vista metodológico, la adopción de **SCRUM** ha sido un elemento clave para mantener el ritmo del proyecto a lo largo de los cuatro meses de trabajo. La división del esfuerzo en sprints cortos ha permitido reaccionar a los problemas conforme aparecían, ajustar la planificación cuando un sprint se complicaba (especialmente el Sprint 3, en el que la integración con MySQL requirió más tiempo del previsto inicialmente) y mantener una visión clara del avance real frente al planificado.

En el plano personal, el equipo destaca como aprendizajes principales: la importancia de una comunicación clara y frecuente entre los miembros, el valor de los code reviews entre compañeros para mejorar la calidad del código, y la utilidad de planificar antes de programar — un buen diagrama de clases ha ahorrado decenas de horas de refactorización posterior. El proyecto ha sido, en definitiva, una experiencia formativa muy completa y una primera aproximación realista al trabajo profesional como desarrollador.

7. Trabajos futuros

Aunque la aplicación entregada cumple los requisitos del enunciado, el equipo identifica varias líneas de mejora y extensión que podrían abordarse en futuras iteraciones o en un eventual segundo curso del ciclo:

- **Migración a arquitectura cliente-servidor:** exponiendo la lógica de negocio mediante una API REST construida con **Spring Boot**.
- **Frontend web:** implementación de una interfaz con HTML/CSS/JavaScript que reemplace o complemente la actual interfaz Swing.
- **Cifrado adaptativo de contraseñas:** uso de **bcrypt** o Argon2 con sal aleatoria por usuario, alineándose con las recomendaciones OWASP.
- **Internacionalización (i18n):** permitir cambiar entre español e inglés en tiempo de ejecución.
- **Notificaciones por correo electrónico:** envío automático a clientes y empleados cuando se cree, modifique o cancele una cita.
- **Panel de informes y estadísticas:** ocupación de talleres, número de citas por empleado y mes, distribución de clientes por tipo (héroe/villano), etc.
- **Integración continua (CI/CD):** con **GitHub Actions** para ejecutar las pruebas JUnit automáticamente en cada push a la rama main.
- **Pruebas de integración y de interfaz:** con AssertJ-Swing o TestFX para validar el comportamiento extremo a extremo de la aplicación.
- **Empaquetado nativo:** distribución como ejecutable autocontenido con **jpackage**, facilitando la instalación sin necesidad de tener el JDK en cada equipo.

Anexos

Anexo I — Listado de requisitos de la aplicación

Se presentan a continuación los requisitos funcionales (RF) y no funcionales (RNF) que han guiado el desarrollo del proyecto. Todos ellos se han verificado durante el Sprint 4 mediante pruebas unitarias y revisión manual.

Requisitos funcionales

ID	Requisito
RF-01	El sistema dispondrá de una pantalla de login con autenticación por apodo y contraseña (hash **SHA-256**).
RF-02	Un Aprendiz solo puede ver las citas en las que participa como aprendiz asignado.
RF-03	Un Oficial puede ver todas las citas, crear nuevas y modificar las que él mismo ha creado.
RF-04	Un Oficial puede asignar hasta 2 aprendices a cada cita que crea.
RF-05	Un Maestro puede ver, crear, modificar y borrar cualquier cita del sistema.
RF-06	El Maestro puede dar de alta, modificar y borrar clientes (superhéroes y supervillanos).
RF-07	El Maestro puede dar de alta, modificar y borrar talleres del estudio.
RF-08	Cada cita tiene cliente, traje, empleado responsable, taller, día, hora y duración (mín. 1 hora).
RF-09	Cada cliente tiene ID único, nombre de superhéroe, superpoder, colores y tipo (Héroe/Villano).
RF-10	Cada traje tiene ID, cliente asociado, nombre y estado de fabricación (Diseño/Costura/Taller).
RF-11	Cada taller tiene ID, nombre de sala (ciudad) y tipo de sala (Diseño/Costura/Pruebas).
RF-12	Cada empleado tiene ID, nombre, apellidos, apodo, categoría (Aprendiz/Oficial/Maestro) y contraseña.
RF-13	BONUS: un héroe y un villano no pueden tener citas en el mismo taller y día con menos de 60 minutos de margen.

Requisitos no funcionales

ID	Requisito
RNF-01	La interfaz se desarrolla con Java Swing siguiendo el patrón MVC.
RNF-02	Los datos se persisten en una base de datos relacional **MySQL 8**.
RNF-03	Las contraseñas se almacenan mediante hash **SHA-256**.
RNF-04	El acceso a la BBDD utiliza **PreparedStatement** para prevenir inyección SQL.

ID	Requisito
RNF-05	El proyecto se versiona en GitHub con commits regulares y ramas feature.
RNF-06	Las clases públicas se documentan con JavaDoc.
RNF-07	El código se acompaña de pruebas unitarias con **JUnit 5**.
RNF-08	El proyecto compila sin errores ni advertencias críticas.
RNF-09	La aplicación debe abrirse en menos de 3 segundos en un equipo de gama media.
RNF-10	La interfaz será coherente visualmente (mismo Look & Feel del sistema operativo).

Anexo II — Guía de uso de la aplicación

Esta guía describe brevemente el flujo de uso típico de la aplicación. Una versión más extensa, con capturas de pantalla, está publicada en la wiki del repositorio de GitHub.

Requisitos previos

- Java Runtime Environment (JRE) 17 o superior.
- Servidor **MySQL 8**.0 en ejecución (por defecto, localhost:3306).
- Base de datos edna_moda creada mediante el script src/SegundoSprint/bd/edna_moda.sql.

Inicio de sesión

Al arrancar la aplicación (clase MainApp) se muestra la ventana de login. Para acceder, introduzca su apodo y contraseña. Los datos de prueba incluidos en el script SQL permiten autenticarse, por ejemplo, como Maestro con apodo EdnaM y contraseña 1234, como Oficial con IronTailor / 1234, o como Aprendiz con AgujaRápida / 1234.

Ventana principal

Tras autenticarse, se abre la ventana principal con una barra de menú cuyas opciones se habilitan en función del rol del empleado. La barra de estado inferior muestra el apodo y la categoría del usuario activo.

Gestión de citas

Desde el menú Citas se accede a la ventana CitasVista. Muestra una tabla con todas las citas (o solo las del usuario, si es Aprendiz). El panel lateral permite crear una nueva cita rellenando los campos cliente, traje, taller, día, hora, duración y empleado responsable, y, opcionalmente, marcar hasta dos aprendices. El sistema valida automáticamente la duración, los solapes y la regla bonus antes de persistir la cita.

Gestión de clientes y talleres

Estas opciones están reservadas al Maestro. Permiten realizar altas, modificaciones y bajas tanto de clientes (nombre de superhéroe, superpoder, colores, tipo) como de talleres (nombre de sala y tipo).

Cierre de sesión y salida

El menú Sesión → Cerrar sesión devuelve al usuario a la pantalla de login. La opción Sesión → Salir termina la ejecución de la aplicación, cerrando previamente cualquier conexión abierta a la base de datos.

Anexo III — Diagramas y diseño técnico

Los diagramas técnicos del proyecto (casos de uso, clases UML, E-R) se han incluido en el cuerpo de la memoria en las secciones 4.1 y 4.5. Los ficheros originales (PNG, PUML y MERMAID) se encuentran en la raíz del repositorio del proyecto.

Anexo IV — Capturas de pantalla de la aplicación

Se incluyen a continuación las capturas más representativas de la aplicación, agrupadas por funcionalidad. Las imágenes originales están disponibles en la carpeta docs/screenshots del repositorio.

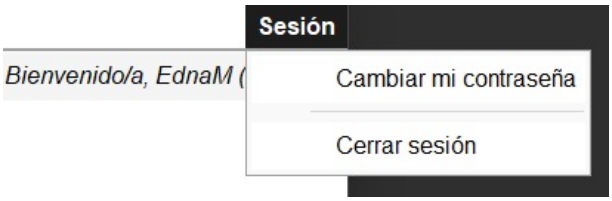
IV.1. Acceso al sistema

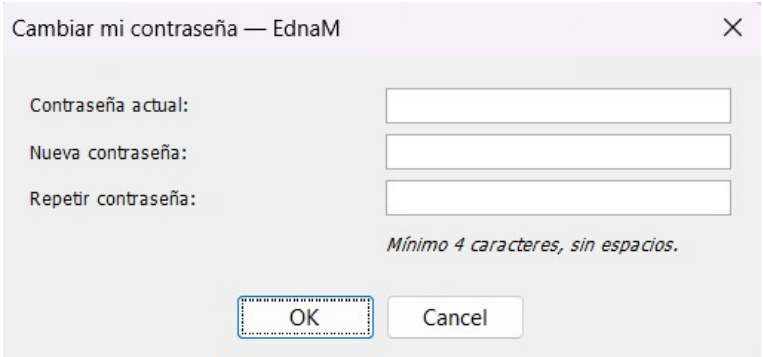
Pantalla inicial de inicio de sesión. El usuario introduce su apodo y contraseña; el sistema verifica las credenciales contra el hash SHA-256 almacenado y, si son correctas, abre la ventana principal cargando el menú según el rol.

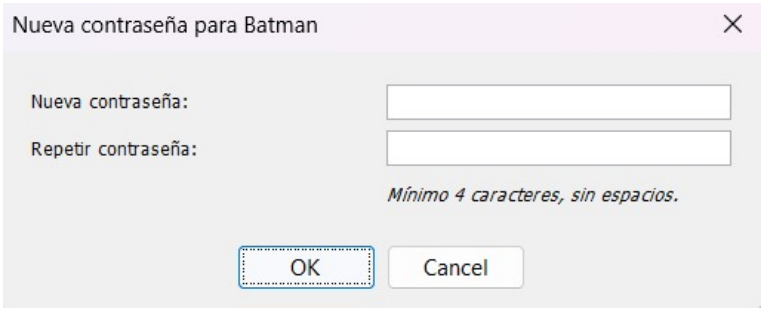


IV.2. Sesión del usuario

Menú Sesión accesible desde la ventana principal, con las opciones "Cambiar mi contraseña" y "Cerrar sesión". El diálogo de cambio de contraseña exige la contraseña actual, la nueva y su confirmación, con un mínimo de 4 caracteres sin espacios. El Maestro también puede restablecer la contraseña de otro empleado al valor por defecto.

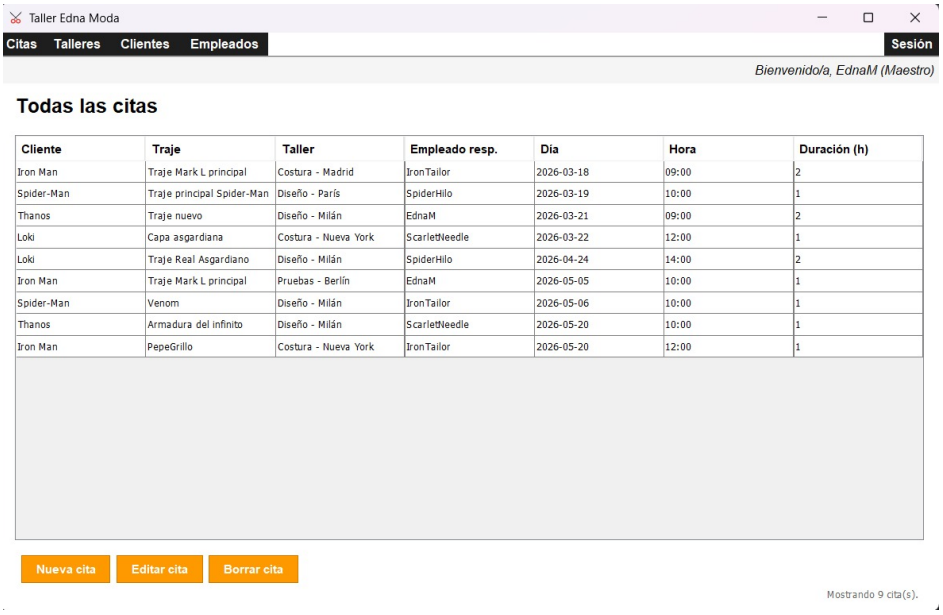




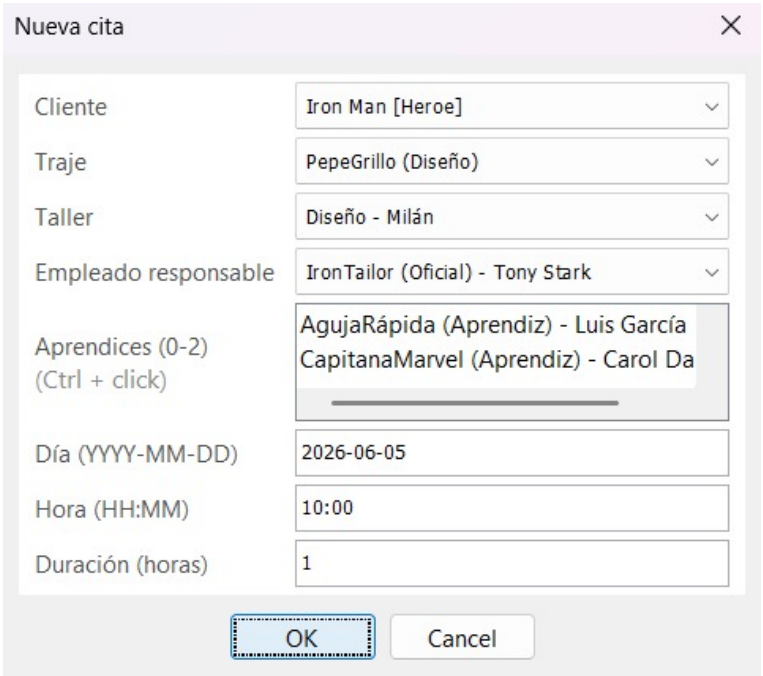

Nueva contraseña para Batman
Nueva contraseña:
Repetir contraseña:
Mínimo 4 caracteres, sin espacios.
OK
Cancel

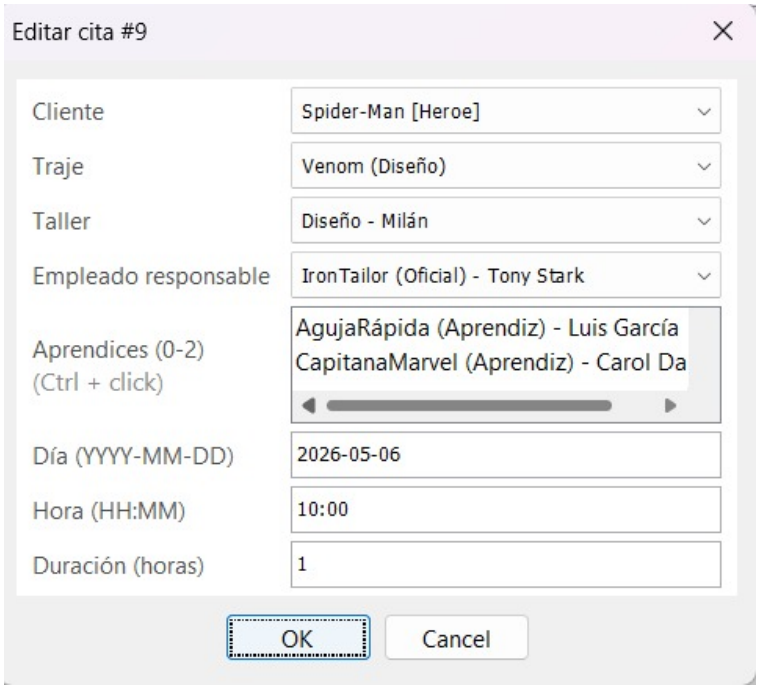
IV.3. Gestión de citas

Vista principal de citas, accesible para los tres roles. El Aprendiz ve solo las suyas; el Oficial y el Maestro ven todas. Los botones de la parte inferior permiten crear, editar o borrar citas según el rol. Los diálogos de alta y edición permiten seleccionar cliente, traje, taller, empleado responsable, aprendices, día, hora y duración.



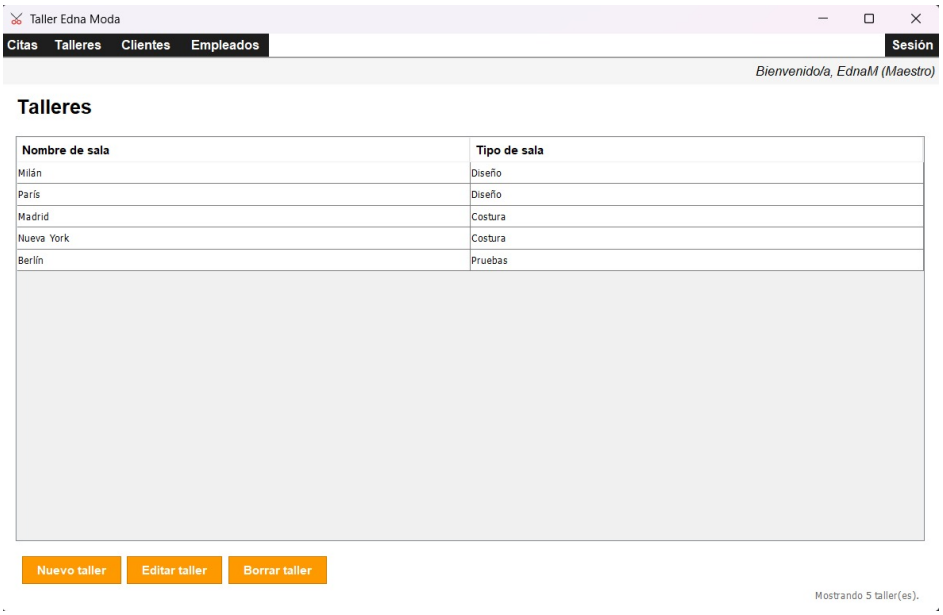
Cliente	Traje	Taller	Empleado resp.	Dia	Hora	Duración (h)
Iron Man	Traje Mark L principal	Costura - Madrid	IronTailor	2026-03-18	09:00	2
Spider-Man	Traje principal Spider-Man	Diseño - París	SpiderHilo	2026-03-19	10:00	1
Thanos	Traje nuevo	Diseño - Milán	EdnaM	2026-03-21	09:00	2
Loki	Capa asgardiana	Costura - Nueva York	ScarletNeedle	2026-03-22	12:00	1
Loki	Traje Real Asgardiano	Diseño - Milán	SpiderHilo	2026-04-24	14:00	2
Iron Man	Traje Mark L principal	Pruebas - Berlín	EdnaM	2026-05-05	10:00	1
Spider-Man	Venom	Diseño - Milán	IronTailor	2026-05-06	10:00	1
Thanos	Armadura del infinito	Diseño - Milán	ScarletNeedle	2026-05-20	10:00	1
Iron Man	PepeGrillo	Costura - Nueva York	IronTailor	2026-05-20	12:00	1

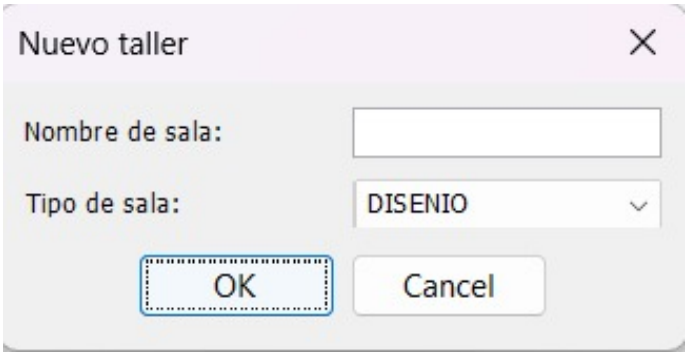


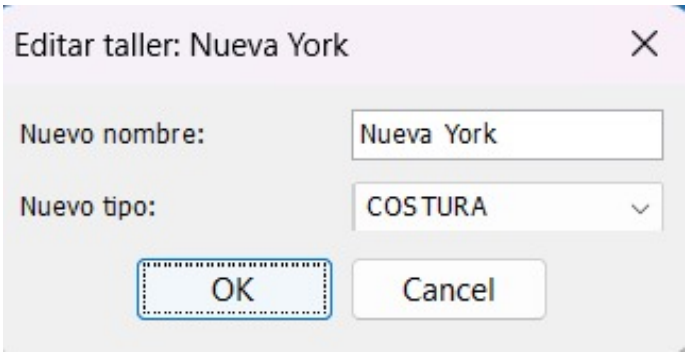

Editar cita #9
Cliente
Spider-Man [Heroe]
Traje
Venom (Diseño)
Taller
Diseño - Milán
Empleado responsable
IronTailor (Oficial) - Tony Stark
Aprendices (0-2)
(Ctrl + click)
AgujaRápida (Aprendiz) - Luis García
CapitanaMarvel (Aprendiz) - Carol Da
Día (YYYY-MM-DD)
2026-05-06
Hora (HH:MM)
10:00
Duración (horas)
1
OK
Cancel

IV.4. Gestión de talleres

Listado de talleres del estudio (Milán, París, Madrid, Nueva York, Berlín…) con su tipo (Diseño, Costura, Pruebas). El Maestro puede dar de alta nuevos talleres, editar nombre y tipo, o eliminarlos cuando ya no se utilicen.

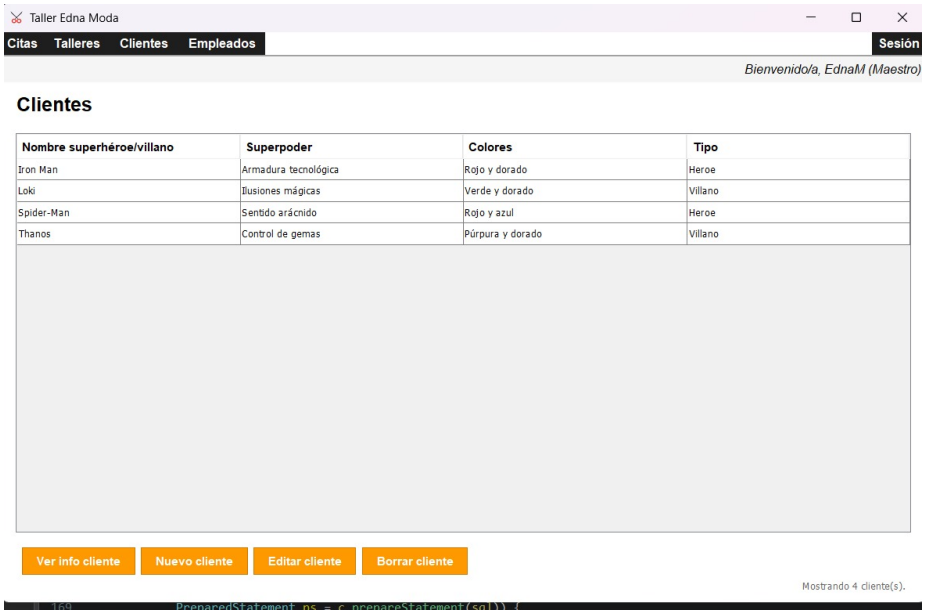


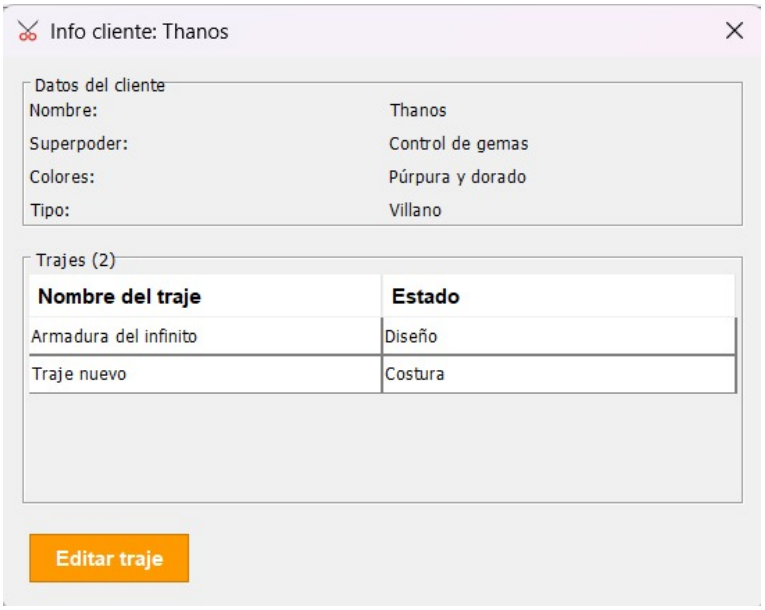


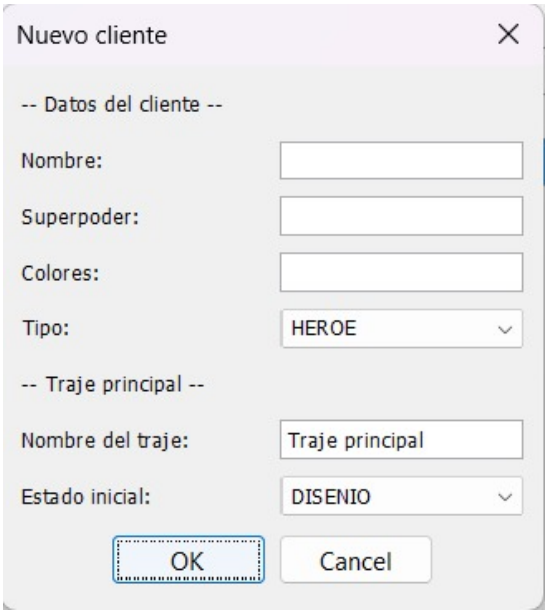


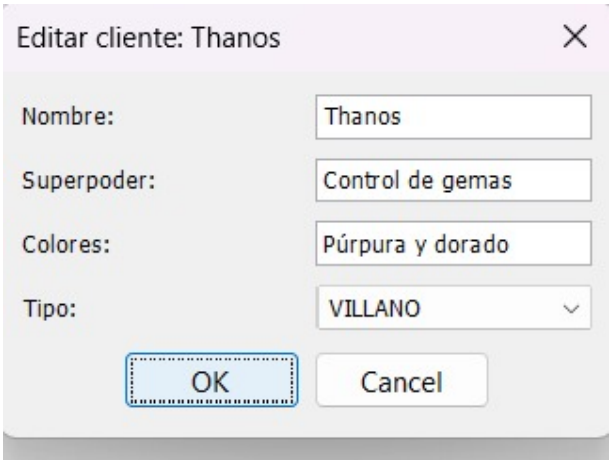
IV.5. Gestión de clientes y trajes

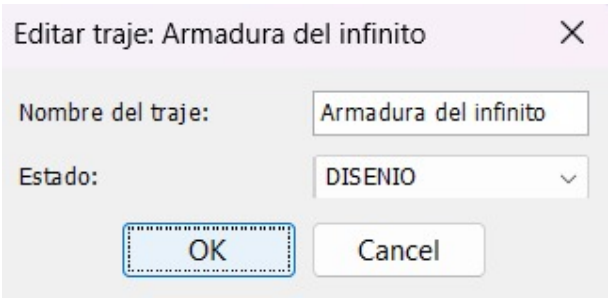
Listado de clientes (héroes y villanos) y ficha individual donde se muestran sus trajes y el estado de cada uno (Diseño/Costura/Pruebas). El Maestro tiene acceso al CRUD completo; el Oficial puede cambiar el estado de un traje según avanza el trabajo.





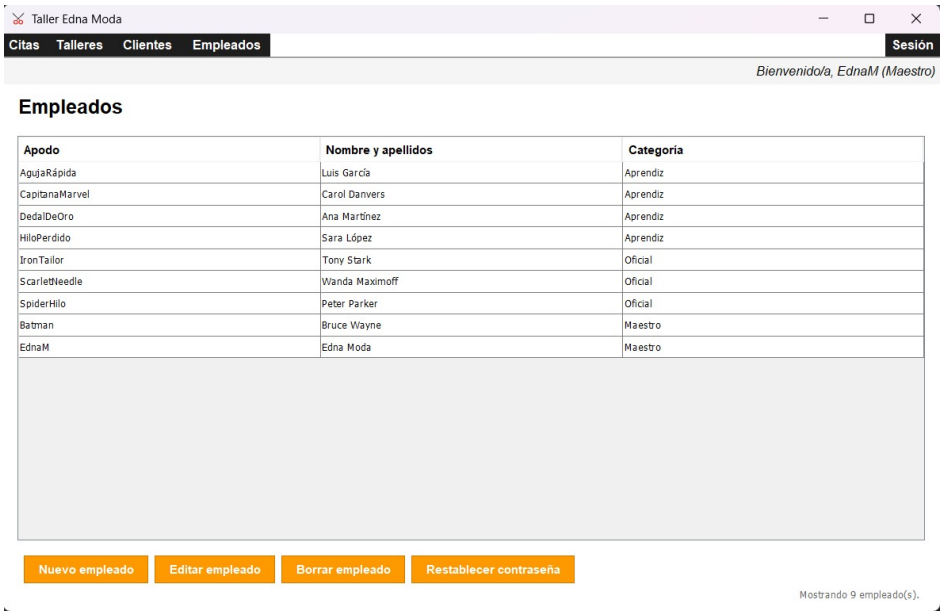

Nuevo cliente
-- Datos del cliente --
Nombre:
Superpoder:
Colores:
Tipo:
HEROE
-- Traje principal --
Nombre del traje:
Traje principal
Estado inicial:
DISENIO
OK
Cancel


Editar cliente: Thanos
Nombre:
Thanos
Superpoder:
Control de gemas
Colores:
Púrpura y dorado
Tipo:
VILLANO
OK
Cancel

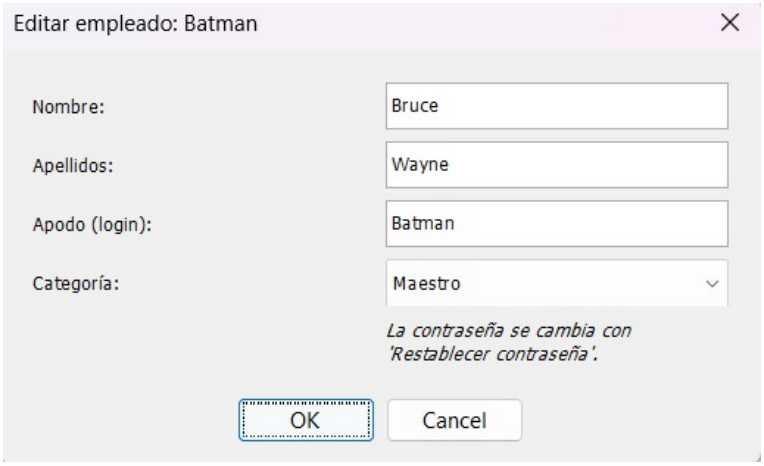

Editar traje: Armadura del infinito
Nombre del traje:
Armadura del infinito
Estado:
DISENIO
OK
Cancel

IV.6. Gestión de empleados

Vista accesible únicamente al Maestro. Permite el alta, edición y baja de empleados, así como restablecer la contraseña al valor por defecto. La tabla muestra apodo, nombre completo y categoría de cada miembro del taller. La aplicación bloquea acciones peligrosas (borrarse a sí mismo, dejar el sistema sin Maestros, eliminar a EdnaM).






Editar empleado: Batman
Nombre:
Bruce
Apellidos:
Wayne
Apodo (login):
Batman
Categoría:
Maestro
La contraseña se cambia con 'Restablecer contraseña'.
OK
Cancel